

ITEC 116: System Integration and Architecture 2

Group Activity 5: Execution & Implementation (Code Base Continuation)

Node A: SCM Engineers (Outgoing Webhook Event)

1. Validated JSON Payload (Shipment Delayed Event)

This payload represents the standardized webhook event emitted by the SCM application upon a state change.

```
{
  "event_id": "evt_94857392_scm",
  "eventType": "Shipment_Delayed",
  "timestamp": "2026-09-22T20:39:26Z",
  "data": {
    "shipmentId": "SHP-10024",
    "project_id": "PRJ-5501",
    "newStatus": "Delayed",
    "delayReason": "Customs clearance hold",
    "estimatedDelivery": "2026-09-28"
  }
}
```

2. Documented REST API Specifications (Publish Node)

- **Endpoint:** POST <https://middleware-broker.internal/webhooks/scm-events>
- **Authentication:** Bearer Token (JWT)
- **Headers:** Content-Type: application/json, X-Idempotency-Key: <event_id>
- **Success Response:** 202 Accepted - The middleware successfully received the webhook for processing.

Node B: PM Engineers (Incoming REST API Endpoint)

1. Documentation of Incoming REST API Contracts

The PM system exposes this endpoint to accept transformed data from the middleware to update project tasks.

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "properties": {
    "projectId": { "type": "string" },
    "taskId": { "type": "string" },
    "taskStatus": { "type": "string", "enum": ["Active", "On_Hold", "Completed"] },
    "impactNotes": { "type": "string" }
  },
  "required": ["projectId", "taskStatus", "impactNotes"]
}
```

2. Executable Logic (Python/Flask)

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route('/api/v1/projects/<projectId>/tasks/status-update', methods=['POST'])
def update_task_status(projectId):
    payload = request.get_json()

    # Validation
    if not payload or 'taskStatus' not in payload or 'impactNotes' not in payload:
        return jsonify({"error": "Missing required fields"}), 400
```

```

# Process Transformed Data
if payload['taskStatus'] == "On_Hold":
    # Database execution logic would go here
    notes = payload['impactNotes']
    print(f"Project {projectId} placed On_Hold. Reason: {notes}")
    return jsonify({"status": "Success", "message": "Task updated"}), 200

if __name__ == '__main__':
    app.run(port=5000)

```

Node C: Architects (Middleware Node C)

1. Complete Data Field Mapping Matrix

Based on the provided database schema screenshot, the middleware will enforce the following data type casting and mapping matrix:

Database Name	Type	JSON key	Target Type
patient_num	VARCHAR(50)	patient.id	String
f_name	VARCHAR(50)	patient.fname	String
l_name	VARCHAR(50)	patient.lname	String
date	DATE(MM/DD/YY)	patient.birth	String
is_confirmend	INT(1)	appointment.confirm	Boolean
provider_code	VARCHAR(50)	appointment_provider.id	String
appt_date	DATETIME	appointment.date	String
service_list	VARCHAR(DELIMITED)	requestedService[]	String
clinical_flag	INT(1)	accessControl.clinic	Boolean
contact_num	VARCHAR(20)	patientcontact.number	String

2. Asynchronous Resiliency & Transformation Logic (Node.js)

This logic utilizes an Idempotency Key (event_id) to prevent duplicate operations and includes a retry strategy.

```

const processedEvents = new Set(); // Distributed cache for idempotency

async function processSCMEvent(eventPayload, retryCount = 0) {
    const MAX_RETRIES = 3;
    const eventId = eventPayload.event_id;

    // 1. Idempotency Check
    if (processedEvents.has(eventId)) {
        console.log(`Event ${eventId} already processed. Skipping duplicate.`);
        return;
    }

    try {
        // 2. Transformation Logic
        const transformedData = {
            projectId: eventPayload.data.project_id,
            taskId: "AUTO-DETECT",
            taskStatus: eventPayload.data.newStatus === "Delayed"
                ? "On_Hold" : "Active",
            impactNotes: `Shipment ${eventPayload.data.shipmentId} delayed: ` +
                `${eventPayload.data.delayReason}`
        };

        // 3. Forward to PM System (Node B)
    }
}

```

```

        await sendToPMSystem(transformedData);

        // Mark as processed upon success
        processedEvents.add(eventId);
        console.log("Integration successful and ACID transaction committed.");
    } catch (error) {
        // Asynchronous Resiliency & Rollback
        console.error(`Transmission failed: ${error.message}`);
        if (retryCount < MAX_RETRIES) {
            console.log(`Retrying... Attempt ${retryCount + 1}`);
            setTimeout(() => processSCMEvent(eventPayload, retryCount + 1),
                2000 * (retryCount + 1)); // Exponential backoff
        } else {
            console.log("Max retries reached. Routing to Dead Letter Queue (DLQ).");
            routeToDLQ(eventPayload, error);
        }
    }
}

```

Node D: PM & QA

1. Integration Post-Deployment KPIs

- **Integration Success Rate (ISR):** Percentage of SCM webhook events successfully transformed and committed to the PM system without requiring the Dead Letter Queue. Target: > 99%.
- **Average Processing Latency:** The total time elapsed (in milliseconds) from the SCM webhook emission to the final HTTP 200 OK response from the PM system. Target: < 500ms.
- **Idempotency Rejection Rate:** The frequency of duplicate event_id payloads intercepted by the middleware layer, indicating network retries or SCM system anomalies.

2. Activity Milestone Timeline

- **Milestone 1 (API Contracts & Payloads):** Teams A and B finalize standard identifiers (JSON specs) and lock API endpoint definitions.
- **Milestone 2 (Middleware & Mapping Execution):** Team C completes the functional field mapping matrix and programs the asynchronous retry and DLQ logic.
- **Milestone 3 (Integration Audit & Final Deployment):** Team D executes full end-to-end testing, confirming functional design against post-deployment KPIs and validating system rollbacks on simulated network failures.